

Semana 1: Diseño conceptual y generación de datos



Noelia Reyes Rodríguez
Nicole Álvarez Estévez
Yoana Álvarez Estévez
Aníbal Alberto Díaz
Sergio García Afonso

Índice

Semana 1: Diseño conceptual y generación de datos.....	1
Índice.....	2
Repartición de funciones.....	3
Introducción.....	3
Diseño Conceptual y Lógico de Datos.....	4
1. Creación del diagrama.....	4
2. Creación del modelo relacional.....	5
3. Creación de la base de datos.....	5
4. Crear usuario.....	8
Desarrollo del Motor de Generación de Datos Sintéticos.....	9
1. Generar ID aleatoria.....	9
2. Cursor.....	10
3. Conectar base de datos.....	10
4. Constantes.....	11
5. Generar departamentos y operadores.....	12
6. Generar clientes.....	13
7. Generar ticket y mensaje.....	14
8. Escribir log.....	15
9. Seeder.....	15

Repartición de funciones

Base de Datos: Nicole y Aníbal

Script en Python: Yoana y Sergio

Ayuda en el Script y subir a GitHub: Noelia

Documentación e informe: Noelia

Introducción

En este documento presentamos Hermes IT, un sistema de base de datos diseñado para gestionar un entorno de soporte técnico con clientes, operadores, departamentos, tickets y mensajes.

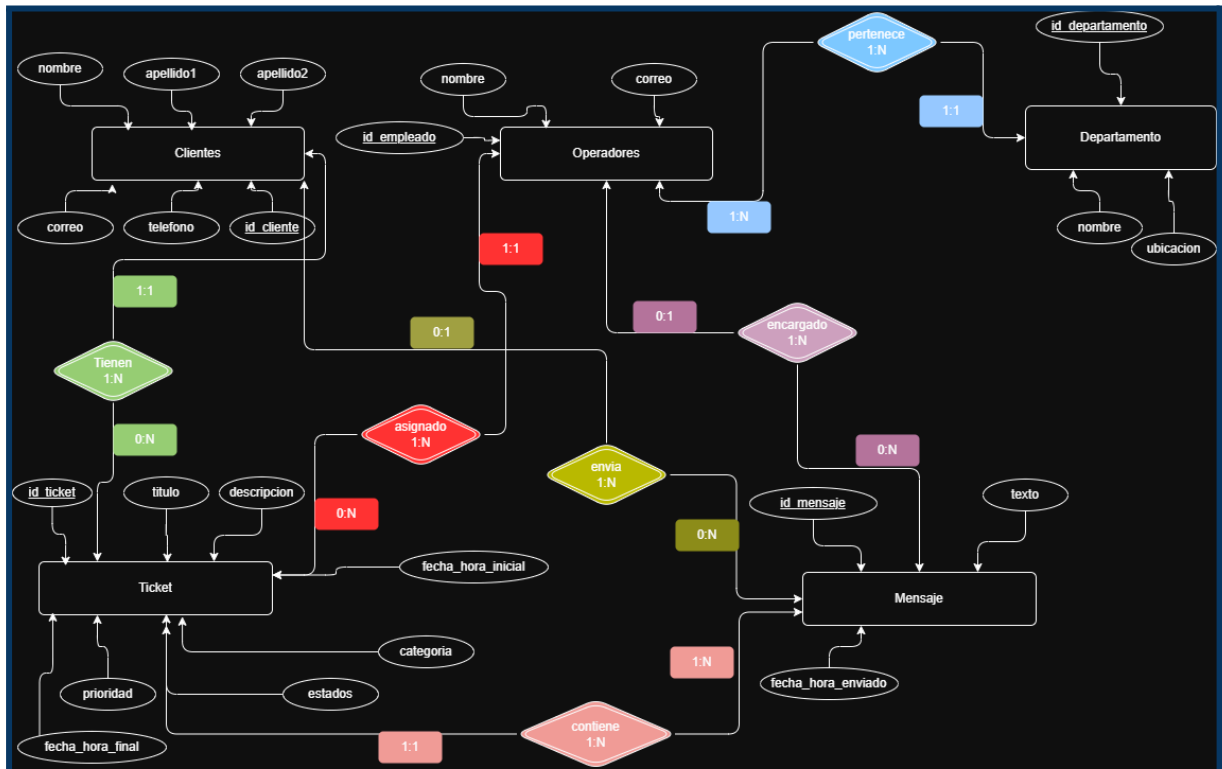
El proyecto consta de un diseño conceptual mediante un diagrama, a partir del cual se creó la base de datos en MySQL.

Posteriormente, se desarrolló un programa de generación de datos automáticos en Python utilizando la librería Faker, respetando las restricciones del modelo relacional.

Este proyecto ha sido un trabajo en equipo, dividiendo las tareas entre los miembros del equipo en el diseño de la base de datos y la implementación de los datos en el script de Python.

Diseño Conceptual y Lógico de Datos

1. Creación del diagrama



Creamos un diagrama inicial de nuestra base de datos con draw.io para tener un esquema inicial.

La relación de Cientes con Ticket es una relación $1:1 \rightarrow 0:N$, ya que un cliente puede tener de cero a muchos tickets, pero cada ticket pertenece a exactamente un cliente.

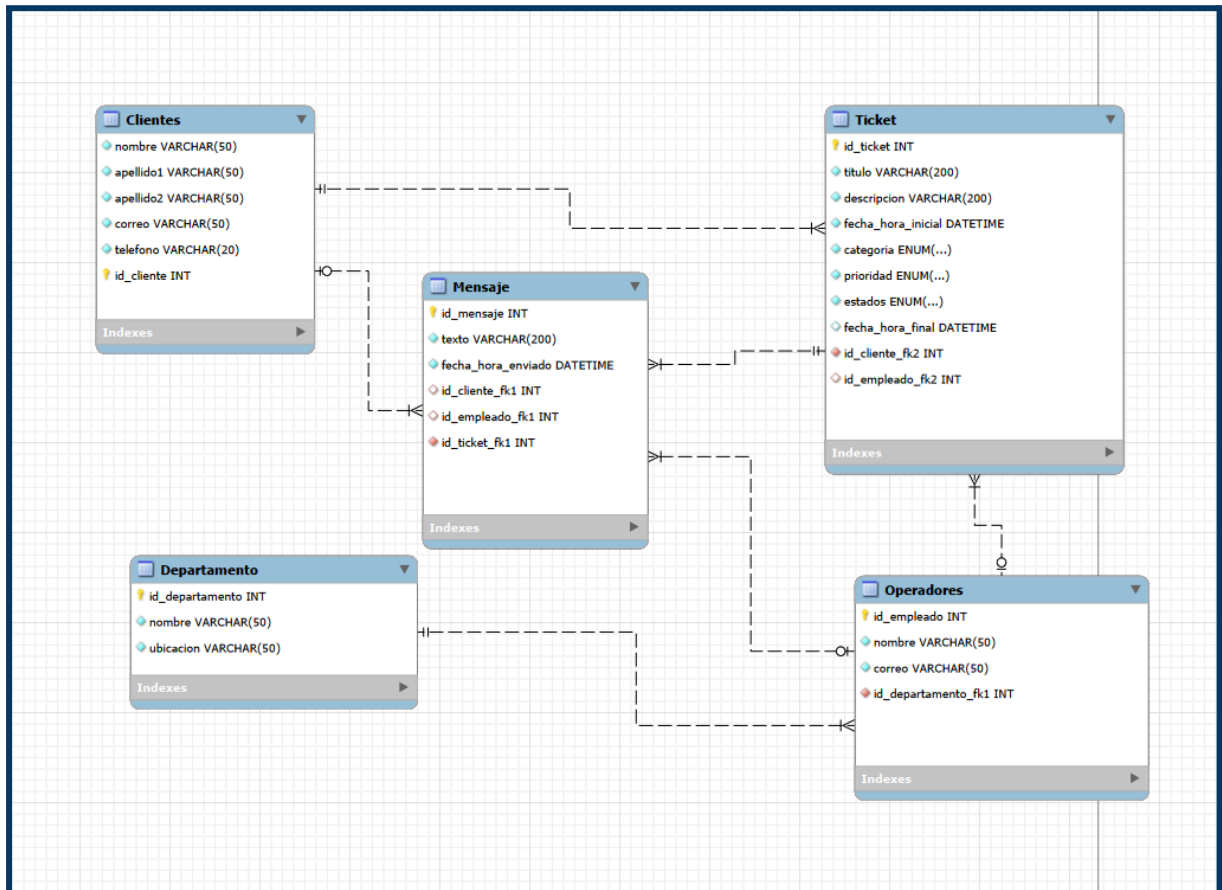
La relación de Operadores con Ticket tiene una relación de $1:1 \rightarrow 0:N$, debido a que un ticket puede tener asignado cero o un operador, y un operador puede tener asignados cero o muchos tickets.

La relación de Operadores con Mensaje es una relación $0:1 \rightarrow 0:N$, por que un mensaje puede haber sido enviado por cero o un operador y un operador puede haber enviado cero o muchos mensajes.

La relación de Operadores con Departamento cuenta con una relación $1:N \rightarrow 1:1$, ya que un departamento tiene muchos operadores, pero cada operador pertenece a exactamente un departamento.

La relación de Ticket con Mensaje tiene una relación de $1:1 \rightarrow 1:N$, debido a que un ticket contiene muchos mensajes, y cada mensaje pertenece a exactamente un ticket.

2. Creación del modelo relacional



Después de acabar el diagrama y tener claro las tablas, las relaciones y el tipo de relación que iba a tener cada una, construimos el modelo entidad-relación siguiendo esas bases.

3. Creación de la base de datos

Creamos la base de datos, la cual decidimos llamar "Hermes_IT".

```
CREATE DATABASE IF NOT EXISTS Hermes_IT;  
USE Hermes_IT;
```

```
CREATE TABLE Clientes (
    id_cliente INT,
    nombre VARCHAR(50) NOT NULL,
    apellido1 VARCHAR(50) NOT NULL,
    apellido2 VARCHAR(50) NOT NULL,
    correo VARCHAR(50) NOT NULL UNIQUE,
    telefono VARCHAR(20) NOT NULL,
    PRIMARY KEY(id_cliente)
)
ENGINE = InnoDB;
```

Empezamos creando la tabla Clientes añadiendo un ID a cada cliente como Primary Key. Al email le pusimos "UNIQUE" para que no se puedan repetir y en teléfono decidimos ponerle un VARCHAR de 20, ya que cuando pasamos al script de Python nos dimos cuenta que la función "fake.number()" nos podía dar como máximo 20 caracteres.

```
CREATE TABLE Departamento (
    id_departamento INT NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    ubicacion VARCHAR(50) NOT NULL,
    PRIMARY KEY(id_departamento)
)
ENGINE = InnoDB;
```

Le sigue la tabla Departamento, donde se va a registrar todos los departamentos con su ubicación y su identificador siendo la Primary Key.

```
CREATE TABLE Operadores (
    id_empleado INT NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    correo VARCHAR(50) NOT NULL UNIQUE,
    id_departamento_fk1 INT NOT NULL,
    PRIMARY KEY(id_empleado),
    CONSTRAINT FK_departamento_operador
        FOREIGN KEY(id_departamento_fk1)
        REFERENCES Departamento(id_departamento)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
)
ENGINE = InnoDB;
```

La tabla Operadores tiene su ID como Primary Key y, además, al estar relacionada con la tabla Departamento, tiene el ID del departamento como Foreign Key. Tanto la ID de empleado como la ID de cliente no están con la restricción en cascada porque más adelante se usa en un CHECK() y MySQL no permite restricciones en cascada para columnas que se usen en un CHECK().

Decidimos poner ON DELETE RESTRICT en la tabla Operadores te obliga a reubicar a los operadores antes de eliminar el departamento.

```

CREATE TABLE Ticket(
    id_ticket INT NOT NULL,
    titulo VARCHAR(200) NOT NULL,
    descripcion VARCHAR(200) NOT NULL,
    fecha_hora_inicial DATETIME NOT NULL,
    categoria ENUM('Hardware','Software','Redes','Seguridad') NOT NULL,
    prioridad ENUM('Baja','Media','Alta','Critica') NOT NULL,
    estados ENUM('Abierto','En proceso','Cerrado','Archivado') NOT NULL,
    fecha_hora_final DATETIME,
    id_cliente_fk2 INT NOT NULL,
    id_empleado_fk2 INT,
    PRIMARY KEY(id_ticket),
    CONSTRAINT FK_empleado_ticket
        FOREIGN KEY(id_empleado_fk2)
        REFERENCES Operadores(id_empleado)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_cliente_ticket
        FOREIGN KEY(id_cliente_fk2)
        REFERENCES Clientes(id_cliente)
        ON DELETE CASCADE
        ON UPDATE CASCADE
)
ENGINE = InnoDB;

```

En la tabla Ticket tuvimos que usar varios ENUM para enumerar los posibles valores que podían haber en categoría, prioridad y estados. Ticket está relacionada con la tabla Operadores y con la tabla Clientes y tiene, como Primary Key, la propia ID del ticket.

```

CREATE TABLE Mensaje(
    id_mensaje INT NOT NULL,
    texto VARCHAR(200) NOT NULL,
    fecha_hora_enviado DATETIME NOT NULL,
    id_cliente_fk1 INT,
    id_empleado_fk1 INT,
    id_ticket_fk1 INT NOT NULL,
    PRIMARY KEY(id_mensaje),
    CONSTRAINT CHK_autoria_mensaje CHECK (
        (id_cliente_fk1 IS NOT NULL AND id_empleado_fk1 IS NULL)
        OR
        (id_cliente_fk1 IS NULL AND id_empleado_fk1 IS NOT NULL)
    ),
    CONSTRAINT FK_ticket_mensaje
        FOREIGN KEY(id_ticket_fk1)
        REFERENCES Ticket(id_ticket)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_empleado_mensaje
        FOREIGN KEY(id_empleado_fk1)
        REFERENCES Operadores(id_empleado),
    CONSTRAINT FK_mensaje_cliente
        FOREIGN KEY(id_cliente_fk1)
        REFERENCES Clientes(id_cliente)
)
ENGINE = InnoDB;

```

Por último, creamos la tabla Mensaje donde su Primary Key es la ID del mensaje y está relacionada con la tabla Ticket, la tabla Operadores y la tabla Cliente.

Hicimos un CONSTRAINT CHECK para saber quien es la persona que escribió. Si el ID del cliente no es NULL y el ID del empleado si lo es, entonces asumimos que quien está escribiendo el mensaje es un cliente pidiendo información. En cambio, si es el ID del cliente el que está NULL y el del empleado no, asumimos que quien escribe es un operador comunicando una solución.

4. Crear usuario

```
CREATE USER IF NOT EXISTS 'hermesadmin'@'localhost' IDENTIFIED BY '1234';  
GRANT ALL PRIVILEGES ON *.* TO 'hermesadmin'@'localhost' WITH GRANT OPTION;  
FLUSH PRIVILEGES;
```

Creamos un nuevo usuario y le pusimos como nombre "hermesadmin", el cual solo puede conectarse de manera local, con la contraseña "1234".

A este usuario le dimos todos los permisos posibles sobre la base de datos y todas las tablas y, además, tiene permisos de administrador por lo que puede darle permisos a otros usuarios.

Desarrollo del Motor de Generación de Datos Sintéticos

1. Generar ID aleatoria

```
def generar_id(lista_id):  
    """  
    Genera un id aleatorio de 6 cifras  
    Args:  
        lista_id: es la lista de id donde se van añadiendo para comprobar si esta el id  
    Returns:  
        Un entero entre 100000 y 999999.  
    """  
    id=random.randint(100000,999999)  
    while id in lista_id:  
        id=random.randint(100000,999999)  
    return id
```

Este es un método auxiliar, el cual conseguimos refactorizando nuestro código.

Nos permite conseguir una ID totalmente aleatoria y única de 6 dígitos. Gracias al bucle while, no se puede repetir, porque el bucle sigue generando una ID siempre que esté en la lista.

2. Cursor

```
def cursor(conexion,lista, sql,nombre_tabla):  
    """  
    Hace una insercion grande con executemany ya que le pasamos una lista de tuplas con los datos  
    Args:  
        conexion: conexion SQL  
        lista: lista de tuplas con los datos  
        sql: sentencia SQL  
        nombre_tabla: nombre de la tabla destino para apuntarla en el log.  
    """  
    try:  
        cursor = conexion.cursor()  
        cursor.executemany(sql,lista)  
        conexion.commit()  
        escribir_log(f"INFO: {len(lista)} registros insertados en {nombre_tabla}")  
    except:  
        print("Error al insertar en la base de datos ")
```

La función cursor es otro método auxiliar que extrajimos refactorizando, debido a que se repetía cada vez que queríamos insertar datos.

Recibe una lista de tuplas con los datos a insertar, sql que lo necesitamos para poder insertar los datos (INSERT INTO) y el nombre de la tabla para mandar el mensaje de que todo se ha insertado correctamente. Además, hicimos un try para que, en el caso de dar error, se muestre el error por pantalla y el programa no se rompa.

Llamaremos a esta función de esta manera:

```
cursor(conexion, lista_de_operadores, sql, "OPERADORES")
```

3. Conectar base de datos

```
import mysql.connector
from funcionalidades_aparte_base_datos import escribir_log

def conectar_bd():
    """
    Establece la conexión con MySQL y la devuelve.
    Returns:
        Objeto conexion y si no None.
    """
    try:
        conexion = mysql.connector.connect(
            host="localhost",
            user="hermesadmin",
            password="1234",
            database="Hermes_IT"
        )
        return conexion
    except:
        print("Fallo de conexion")
        escribir_log(f"ERROR: Fallo de conexión")
        return None
```

Esta función nos conecta con la base de datos que hemos hecho previamente, poniendo el nombre de usuario, la contraseña y el nombre de la base de datos.

Si da error manda un mensaje evitando que se rompa el programa.

4. Constantes

```
from faker import Faker

FAKE = Faker('es_ES')
DEPARTAMENTOS = ["Soporte nivel1", "Soporte nivel2", "Administracion", "RRHH"]
NUM_OPERADORES = 20
NUM_CLIENTES = 50
NUM_TICKETS = 200
PRIORIDAD_LISTA = ["Baja", "Media", "Alta", "Critica"]
CATEGORIA_LISTA = ["Hardware", "Software", "Redes", "Seguridad"]
LISTA_DE_ESTADOS = ["Abierto", "En proceso", "Cerrado", "Archivado"]
LISTA_ID_TICKET = []
LISTA_ID_MENSAJE = []
LISTA_DE_TICKET = []
LISTA_DE_MENSAJE = []
```

Hicimos otro archivo donde guardamos todas nuestras constantes para poder utilizarlas en nuestro código principal.

5. Generar departamentos y operadores

```
def generar_departamentos_y_operadores(conexion):
    """
    Crea los departamentos que estan en la constante departamentos y genera los operadores
    Args:
        conexion

    Returns:
        Lista con los IDs de los operadores para despues poderlos asignar a un ticket
    """
    lista_de_departamentos_a_insertar = []
    lista_id = []
    for departamento in DEPARTAMENTOS:
        id_departamento = generar_id(lista_id)
        lista_id.append(id_departamento)
        numero = random.randint(1,5)
        ubicacion = f"Planta {numero}"
        lista_de_departamentos_a_insertar.append((id_departamento,departamento,ubicacion))
    sql = """ INSERT INTO Departamento
        VALUES (%s,%s,%s)
    """
    cursor(conexion,lista_de_departamentos_a_insertar, sql,"DEPARTAMENTO")

    lista_id_operadores = []
    lista_de_operadores = []
    for veces in range(NUM_OPERADORES):
        id_empleado = generar_id(lista_id_operadores)
        lista_id_operadores.append(id_empleado)
        departamento=random.choice(lista_de_departamentos_a_insertar)
        nombre = FAKE.first_name()
        correo = FAKE.unique.email()
        id_departamento_fk1 = departamento[0]
        lista_de_operadores.append((id_empleado,nombre,correo,id_departamento_fk1))

    sql = """ INSERT INTO Operadores
        VALUES(%s,%s,%s,%s)
    """
    cursor(conexion,lista_de_operadores, sql,"OPERADORES")

    return lista_id_operadores
```

Este método itera sobre la lista de departamentos que metimos en la constante anteriormente y para cada uno de ellos les genera una ID con el método "generar_id" y la mete en una lista. Para la ubicación, usamos un randint que nos da un número aleatorio entre el 1 y el 5 y ese número lo metemos en un string junto a la palabra "Planta". Todo esto se mete en una lista de tuplas.

Con los operadores iteramos 20 veces para obtener 20 empleados, elegimos un departamento aleatorio de la función anterior. Tanto para el nombre como para el correo usamos la función "FAKE", que nos genera un dato aleatorio y automático, en el caso del email remarcamos que es único para que no se genere otro repetido, ya que en la base de datos pusimos que fuera un campo único. Utilizamos el mismo id de departamento que el de la función anterior y lo guardamos todo en otra lista.

Ambas listas las insertamos en sus respectivas tablas con el parámetro "sql". Por último, retornamos la lista con las ID de los operadores para después poderlas insertar a los tickets.

6. Generar clientes

```
def generar_clientes(conexion):  
    """  
    Genera los clientes  
    Args:  
        conexion SQL  
  
    Returns:  
        Lista con los Id de los clientes  
    """  
    lista_de_clientes = []  
    lista_id_clientes = []  
    for i in range(NUM_CLIENTES):  
        nombre = FAKE.first_name()  
        apellido1 = FAKE.last_name()  
        apellido2 = FAKE.last_name()  
        correo = FAKE.email()  
        telefono = FAKE.phone_number()  
        id_cliente = generar_id(lista_id_clientes)  
        lista_id_clientes.append(id_cliente)  
        lista_de_clientes.append((id_cliente,nombre,apellido1,apellido2,correo,telefono))  
  
    sql = """ INSERT INTO Clientes  
        VALUES(%s,%s,%s,%s,%s,%s)  
    """  
  
    cursor(conexion,lista_de_clientes, sql,"CLIENTES")  
  
    return lista_id_clientes
```

Para generar los clientes hicimos lo mismo que con el resto de tablas: añadimos los datos automáticamente con la función "FAKE" y llamamos a nuestra función para generar ID, lo metemos todo en una lista de tuplas y lo insertamos en la tabla Clientes.

Nuevamente, retornamos el ID de los clientes para usarlo en la creación de tickets.

7. Generar ticket y mensaje

```
def generar_ticket_y_mensaje(id_cliente,id_operador):
    """
    generamos un ticket que puede ser de un cliente o un operador y que tenga entre 1 o 3 mensajes
    Args:
        id_cliente
        id_operador
    """
    id_ticket = generar_id(LISTA_ID_TICKET)
    LISTA_ID_TICKET.append(id_ticket)
    titulo = FAKE.sentence(nb_words=4)
    descripcion = FAKE.text(max_nb_chars=100)
    estado = random.choice(LISTA_DE_ESTADOS)
    if estado == "Abierto" or estado == "En proceso" or estado == "Archivado":
        fecha_hora_inicial = FAKE.date_time_between(start_date="-2y" , end_date="now")
        fecha_hora_final = None
    else:
        fecha_hora_inicial = FAKE.date_time_between(start_date="-2y" , end_date="now")
        fecha_hora_final = FAKE.date_time_between(start_date=fecha_hora_inicial, end_date="now")
    prioridad = random.choice(PRIORIDAD_LISTA)
    categoria = random.choice(CATEGORIA_LISTA)

    LISTA_DE_TICKET.append((id_ticket,titulo,descripcion,fecha_hora_inicial,categoria,prioridad,estado,fecha_hora_final,id_cliente,id_operador))

    numero_de_mensajes = random.randint(1,3)
    for veces in range(numero_de_mensajes):
        id_mensaje = generar_id(LISTA_ID_MENSAJE)
        LISTA_ID_MENSAJE.append(id_mensaje)
        texto = FAKE.text(max_nb_chars=100)
        fecha_hora_enviado = FAKE.date_time_between(start_date=fecha_hora_inicial, end_date="now")
        quien_habla = random.choice(["cliente","operador"])
        if quien_habla == "cliente" :
            id_cliente_fk1 = id_cliente
            id_empleado_fk1 = None
        else:
            id_cliente_fk1 = None
            id_empleado_fk1 = id_operador

        LISTA_DE_MENSAJE.append((id_mensaje, texto, fecha_hora_enviado, id_cliente_fk1, id_empleado_fk1, id_ticket))
```

Para la creación de un ticket, nuevamente generamos una ID, usamos la función "FAKE" para los datos generados de manera automática y para la variable estado agarramos un estado random de la constante.

Si el estado está "Abierto" o "En proceso" o "Archivado", se debe añadir una fecha inicial, pero no una final. Para automatizar la fecha inicial pusimos que fuera 2 años antes de la fecha actual.

En cambio, si no es ninguno de esos estados, significa que está "Cerrado", por tanto se debe añadir una fecha inicial y otra final.

Tanto la prioridad como la categoría se seleccionan aleatoriamente de los valores de su constante respectiva.

Todo esto se vuelve a guardar en otra lista de tuplas.

Para los mensajes, se vuelve a generar una ID, se automatiza el texto, se usa la fecha actual y se elige de manera random si quien escribe es un cliente o un operador.

Si el redactor del mensaje es un cliente se usa la ID del cliente, sino se usa la ID del operador.

Nuevamente, se añade a una nueva lista de tuplas.

8. Escribir log

```
def escribir_log(mensaje):
    """
    Registra un mensaje en seed_log.txt

    Args:
        mensaje: texto que registro.
    """
    fecha_hora = datetime.now()
    fecha_formateada = fecha_hora.strftime("%Y-%m-%d %H:%M:%S")
    linea = f"[{fecha_formateada}] {mensaje}\n"
    try:
        with open("seed_log.txt", "a") as archivo:
            archivo.write(linea)
    except:
        print("Error no se pudo registrar el fichero")
```

Este método registra un mensaje en el archivo "seed_log.txt", añade la fecha actual y el mensaje registrado.

Si da error, lanza un mensaje informando del error.

9. Seeder

```
import random
from conexion_base_datos import conectar_bd
from constantes import NUM_TICKETS, LISTA_DE_TICKET, LISTA_DE_MENSAJE
from funciones_base_datos import (cursor, generar_departamentos_y_operadores, generar_clientes, generar_ticket_y_mensaje)

if __name__ == "__main__":
    conexion = conectar_bd()
    if conexion is not None:
        lista_operadores = generar_departamentos_y_operadores(conexion)
        lista_id_clientes = generar_clientes(conexion)

        for i in range(NUM_TICKETS):
            id_operador = random.choice(lista_operadores)
            id_cliente = random.choice(lista_id_clientes)
            generar_ticket_y_mensaje(id_cliente, id_operador)

        sql_T = """ INSERT INTO Ticket
        (id_ticket, titulo, descripcion, fecha_hora_inicial, categoria, prioridad, estados, fecha_hora_final, id_cliente_fk2, id_empleado_fk2)
        VALUES(%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
        """
        cursor(conexion, LISTA_DE_TICKET, sql_T, "TICKET")

        sql_M = """ INSERT INTO Mensaje
        (id_mensaje, texto, fecha_hora_enviado, id_cliente_fk1, id_empleado_fk1, id_ticket_fk1)
        VALUES(%s, %s, %s, %s, %s, %s)
        """
        cursor(conexion, LISTA_DE_MENSAJE, sql_M, "MENSAJES")
        conexion.close()
    else:
        print("fallo de conexion")
```

Este es nuestro programa principal, donde llamamos a todas las funciones e insertamos todos los datos en las tablas Ticket y Mensaje. Si no se pudiera conectar escribiría un mensaje de fallo.

Por último, cerramos la conexión.